Technical Summary: Quantum Algorithms for Optimization and Machine Learning

An In-depth Analysis of VQE, QAOA, BF-DCQO, and QWOA

joao.vmcardoso

March 2026

Resumo

This technical summary presents an in-depth analysis of four cutting-edge quantum algorithms, focusing on their applications in combinatorial optimization and feature selection in machine learning. We explore the *Variational Quantum Eigensolver* (VQE) and the *Quantum Approximate Optimization Algorithm* (QAOA) as foundations for variational algorithms. Next, we detail two innovative approaches: the *Bias-Field Digitized Counterdiabatic Quantum Algorithm* (BF-DCQO) and the *Quantum Whale Optimization Algorithm* (QWOA). For each algorithm, we provide a conceptual preamble, mathematical foundations, a detailed explanation of its operation, an analysis of the specific role of quantum gates in its architecture, a step-by-step pseudocode, and a discussion of benefits and challenges. Finally, we present a consolidated comparative analysis.

1 Introduction: Variational Algorithms and the Quest for Optimal Solutions

Current quantum computing is in the NISQ (*Noisy Intermediate-Scale Quantum*) era, characterized by processors with tens to hundreds of qubits, susceptible to noise and without large-scale error correction. In this context, hybrid quantum-classical variational algorithms have emerged as the main strategy for extracting computational advantage from current quantum devices.
The central paradigm of these algorithms is the use of a parameterized quantum circuit (ansatz) to prepare a quantum state. A classical optimizer iteratively adjusts the circuit parameters to minimize the expected value of a Hamiltonian that encodes the problem to be solved. Next, we explore four algorithms that are based on this or other quantum strategies.

2 Algorithm Preamble

Before detailing each algorithm, it is crucial to understand the context in which each one fits:

- **Problem**: All algorithms discussed here aim to solve optimization problems, either by finding the ground state of a Hamiltonian (which maps a combinatorial optimization problem) or by selecting the best subset of features in a dataset.
- **Quantum Resource**: The algorithms explore quantum resources such as superposition, entanglement, and tunneling to navigate complex search spaces more efficiently than classical counterparts.
- **NISQ Challenge**: The design of each algorithm takes into account the limitations of NISQ hardware, seeking circuits with reduced depth and robustness to noise.

3 Encoding Optimization Problems into Hamiltonians

Most of the algorithms discussed in this work operate on Hamiltonians that directly encode combinatorial optimization problems. For a binary problem with variables $x_i \in \{0, 1\}$, it is possible to establish a correspondence with Pauli operators through the transformation

$$x_i = \frac{1 - \sigma_i^z}{2}.$$

This relation allows rewriting classical cost functions as quantum Hamiltonians of the form

$$H = \sum_k c_k P_k,$$

where P_k are strings of Pauli operators belonging to the set

$$P_k \in \{I, X, Y, Z\}^{\otimes n}.$$

Thus, the optimization problem becomes the task of finding the ground state of the corresponding Hamiltonian.

3.1 Expectation Value Estimation

In variational algorithms such as VQE and QAOA, the expected value of the Hamiltonian is estimated through repeated measurements:

$$E(\vec{\theta}) = \sum_k c_k \langle P_k \rangle.$$

Since different Pauli terms may require distinct measurement bases, it is common to group commuting operators to reduce the number of circuit executions.
This measurement cost constitutes one of the main experimental bottlenecks of variational algorithms.

4 Algorithm 1: Variational Quantum Eigensolver (VQE)

4.1 Preamble

VQE is a hybrid algorithm designed to estimate the ground state of a Hamiltonian H. Its original motivation comes from quantum chemistry, where one seeks the ground state energy of molecules. However, its principle is universal: any combinatorial optimization problem that can be mapped to an Ising Hamiltonian can, in principle, be solved by VQE.

4.2 Mathematical Foundations and Ansatz Construction

VQE is based on the variational principle of quantum mechanics:

$$E_0 \leq \frac{\langle \psi(\vec{\theta}) | H | \psi(\vec{\theta}) \rangle}{\langle \psi(\vec{\theta}) | \psi(\vec{\theta}) \rangle}$$

In practice, the Hamiltonian defined here is decomposed as

$$H = \sum_k c_k P_k.$$

The expected energy value is then calculated as

$$E(\vec{\theta}) = \sum_k c_k \langle P_k \rangle.$$

Each operator P_k may require measurements in different bases. For example, to measure a term involving X operators, a Hadamard gate is applied beforehand to transform the measurement basis:

$$HXH = Z.$$

To reduce the total number of circuit executions, operators that commute with each other can be grouped and measured simultaneously. This procedure is known as *measurement grouping* and constitutes an important aspect in the practical implementation of variational algorithms.

Ansatz Types:

- **Hardware-Efficient Ansatz (HEA)**: Optimized for chip topology, uses alternating layers of single-qubit rotations and entanglement gates. **Gates involved**: R_y, R_z, CX in a linear or ring pattern.
- **Unitary Coupled Cluster (UCC)**: Inspired by quantum chemistry, $U(\vec{\theta}) = e^{T(\vec{\theta}) - T^\dagger(\vec{\theta})}$. **Gates involved**: Requires decomposition into $CNOT$ and R_z gates to implement fermionic excitations.
- **Alternating Layered Ansatz**: Similar to QAOA, alternates layers of problem evolution and mixer.

4.3 Algorithm Explanation

VQE operates in a hybrid loop:

1. A parameterized quantum circuit (ansatz) prepares a trial state $|\psi(\vec{\theta})\rangle$.
2. Measurements are performed to estimate the expected value $\langle \psi(\vec{\theta}) | H | \psi(\vec{\theta}) \rangle$.
3. A classical optimizer (e.g., COBYLA, SPSA) uses this information to generate a new set of parameters $\vec{\theta}$ that minimizes the expected value.
4. The process repeats until convergence.

4.3.1 Gradient Calculation

In variational algorithms, optimization can use gradients of the objective function. A widely used method in parameterized quantum circuits is the *parameter-shift rule*.
For a parameter θ, the energy gradient can be estimated through two circuit evaluations:

$$\frac{\partial E(\theta)}{\partial \theta} = \frac{E(\theta + \pi/2) - E(\theta - \pi/2)}{2}.$$

for standard rotations with Pauli generators ($s = \pi/2$).
This method allows efficient calculation of derivatives on quantum hardware without resorting to traditional numerical differentiation.

4.4 Role of Quantum Gates

- **Single-qubit gates (Ry, Rz)**: Perform rotations on the state of each qubit. The angles of these rotations are the parameters $\vec{\theta}$ that the classical optimizer adjusts. They explore the local state space of each qubit.
- **Two-qubit gates (CX, CZ)**: Generate entanglement between qubits. Entanglement is essential to represent quantum correlations present in the ground state of interacting systems, allowing the ansatz to explore a much richer and more complex region of Hilbert space than would be possible with product states.

4.5 Pseudocode (VQE)

1. Initialize parameters $\vec{\theta}$ randomly.
2. Repeat until convergence is achieved:
   (a) Prepare state $|\psi(\vec{\theta})\rangle$ on the quantum processor by applying the gate sequence (layers of rotations and entanglement).
   (b) Measure qubits in the appropriate format to estimate each term of the Hamiltonian H.
   (c) Send the expected value $\langle H \rangle$ to the classical processor.
   (d) The classical optimizer computes a new $\vec{\theta}$ that minimizes $\langle H \rangle$.
3. Return $\vec{\theta}_{\text{final}}$ and the final state $|\psi(\vec{\theta}_{\text{final}})\rangle$ as an approximation to the ground state.

4.6 Specific Challenges of VQE

- **Barren Plateaus**: In very expressive or deep ansatzes, the gradient tends to zero exponentially with the number of qubits, making optimization unfeasible.
- **Shot Noise**: Finite number of measurements introduces statistical noise into the gradient estimate.
- **Expressivity vs. Trainability**: There is a fundamental trade-off between the ansatz's ability to represent complex states and the ease of optimizing it.

4.6.1 Ansatz Structure

The choice of parameterized ansatz directly influences the algorithm's efficiency. Two main types are frequently used:

- **Hardware-efficient ansatz**: circuits built from hardware-native gates, usually composed of alternating layers of single-qubit rotations and entanglement gates.
- **Problem-inspired ansatz**: circuits designed based on the problem or Hamiltonian structure, which may reduce the number of required parameters.

A known difficulty in optimizing these circuits is the phenomenon of *barren plateaus*, regions of parameter space where the gradient of the objective function becomes exponentially small. This

effect can significantly hinder the convergence of variational algorithms in systems with many qubits.

5 Algorithm 2: Quantum Approximate Optimization Algorithm (QAOA)

5.1 Preamble

QAOA can be interpreted as a specialized variational algorithm, whose ansatz is inspired by quantum adiabatic evolution. It is specifically designed for binary combinatorial optimization problems and has a fixed and interpretable circuit structure, with a number of layers p that controls the circuit depth and, ideally, the solution quality.

5.1.1 Relation to Adiabatic Computing

QAOA can be interpreted as a variational discretization of quantum adiabatic computation. In the adiabatic approach, the system's evolution is governed by a time-dependent Hamiltonian:

$$H(t) = (1 - s(t))H_M + s(t)H_C,$$

where H_C represents the problem Hamiltonian and H_M is a mixing Hamiltonian.
In QAOA, this continuous evolution is approximated by a sequence of parameterized unitary operators:

$$U_C(\gamma_k) = e^{-i\gamma_k H_C},$$

$$U_M(\beta_k) = e^{-i\beta_k H_M}.$$

The alternating application of these operators generates a variational circuit whose depth is controlled by the number of layers p.

5.2 Algorithm Explanation

QAOA alternately applies two unitary operators for a determined time:

- **Problem Operator (U_P)**: Evolves under the problem Hamiltonian H_f for time γ: $U_P(\gamma) = e^{-i\gamma H_f}$.
- **Mixer Operator (U_M)**: Evolves under a "mixer"Hamiltonian (usually $H_M = \sum_i \sigma_i^x$) for time β: $U_M(\beta) = e^{-i\beta H_M}$.

The initial state is the uniform superposition of all computational states, $|\psi_0\rangle = |+\rangle^{\otimes n}$, which is the ground state of the mixer. The final circuit is $|\psi(\vec{\gamma}, \vec{\beta})\rangle = U_M(\beta_p)U_P(\gamma_p)\cdots U_M(\beta_1)U_P(\gamma_1)|\psi_0\rangle$. The parameters $\vec{\gamma}, \vec{\beta}$ are classically optimized to minimize $\langle H_f\rangle$, similarly to VQE.

5.3 Detailed QAOA Circuit Analysis

Problem Operator Decomposition: For a Hamiltonian $H_f = \sum_i h_i\sigma_i^z + \sum_{i<j} J_{ij}\sigma_i^z\sigma_j^z$:

$$U_P(\gamma) = \prod_i e^{-i\gamma h_i\sigma_i^z} \prod_{i<j} e^{-i\gamma J_{ij}\sigma_i^z\sigma_j^z}$$

Implementing $e^{-i\gamma J_{ij}\sigma_i^z\sigma_j^z}$ requires:

1. $CNOT_{i,j}$ gate (target = j)
2. $R_z(2\gamma J_{ij})$ gate on qubit j
3. $CNOT_{i,j}$ gate again

This totals **2 CNOT and 1 Rz gate** per two-body interaction per layer.
Detailed Mixer Operator:

$$U_M(\beta) = \prod_i e^{-i\beta\sigma_i^x} = \prod_i R_x(2\beta)_i$$

Circuit Depth: For n qubits and m interactions, each layer p requires:

- $O(m)$ CNOT gates (limited by hardware connectivity)
- $O(n)$ single-qubit gates (R_x and R_z)

5.4 Role of Quantum Gates

- **Gates implementing** $U_P(\gamma)$: These gates apply conditional phases to the qubit state. Since H_f is diagonal in the computational basis, $U_P(\gamma) = e^{-i\gamma H_f}$ can be decomposed into R_z phase gates on each qubit (for local fields) and R_{zz} gates (or sequences of CNOTs and R_z) for two-body interactions.
- **Gates implementing** $U_M(\beta)$: Since $H_M = \sum_i \sigma_i^x$, $U_M(\beta)$ is simply a rotation around the X-axis on each qubit, implemented by $R_x(2\beta)$ gates.
- **Joint role**: U_P "marks"good solutions with more favorable phases, while U_M explores the search space, creating superposition and transferring amplitude to the marked states.

5.5 Pseudocode (QAOA)

1. Initialize parameters $\vec{\gamma}, \vec{\beta}$ (e.g., randomly).
2. Define circuit depth p.
3. Repeat until convergence:
   (a) Prepare initial state $|+\rangle^{\otimes n}$.
   (b) For $i = 1$ to p:
      i. Apply problem operator $U_P(\gamma_i)$ (implemented with R_z and R_{zz} gates).
      ii. Apply mixer operator $U_M(\beta_i)$ (implemented with R_x gates).
   (c) Measure all qubits to obtain a sample of candidate solutions (bitstrings).
   (d) Calculate the expected value $\langle H_f \rangle$ from the samples.
   (e) Classical optimizer adjusts $\vec{\gamma}, \vec{\beta}$ to minimize $\langle H_f \rangle$.
4. Return the best found bitstring.

5.6 Farhi-Goldstone-Gutmann Theorem

QAOA with $p \to \infty$ recovers adiabatic evolution and converges to the optimal solution. For finite p, the algorithm seeks the best approximation within the circuit's expressive capacity.

5.7 QAOA Challenges

- **Optimization Landscape**: The cost surface can be rugged, with many local minima.
- **Initial Parameter Selection**: Strategies such as parameter interpolation from previous layers ("parameter transfer"technique) can improve convergence.
- **Depth vs. Performance**: Increasing p theoretically improves the solution but introduces more noise and decoherence.

6 Algorithm 3: Bias-Field Digitized Counterdiabatic Quantum Algorithm (BF-DCQO)

6.1 Preamble

BF-DCQO is an evolution of the quantum adiabatic computation paradigm. It seeks to accelerate the inherently slow adiabatic process through the engineering of "counterdiabatic"Hamiltonians that suppress unwanted transitions between energy levels. The digitized version enables its implementation on gate-based quantum computers, and the iterative loop with bias fields (BF) adds a machine learning layer to the process, guiding the system toward high-quality solutions.

6.2 HUBO Problems

The BF-DCQO algorithm is particularly suitable for solving higher-order binary optimization problems, known as *Higher-Order Unconstrained Binary Optimization* (HUBO).
A HUBO Hamiltonian can be expressed as

$$H = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j + \sum_{i,j,k} K_{ijk} Z_i Z_j Z_k + \cdots$$

where terms can involve interactions of arbitrary order between qubits.
Unlike many QUBO-based approaches, BF-DCQO allows directly handling higher-order interactions without needing to introduce additional auxiliary qubits.

6.3 In-depth Counterdiabatic Foundation

The exact adiabatic gauge potential is given by:

$$A_\lambda = i\hbar \sum_{n \neq m} \frac{\langle n(\lambda) | \partial_\lambda m(\lambda) \rangle}{E_m - E_n} |n(\lambda)\rangle\langle m(\lambda)|$$

Nested Commutator Expansion: For the first-order approximation used:

$$A_\lambda^{(1)} = i\alpha(\lambda)[H_{ad}(\lambda), \partial_\lambda H_{ad}(\lambda)]$$

For a Hamiltonian of the form $H_{ad}(\lambda) = (1-\lambda)H_i + \lambda H_f$:

$$\partial_\lambda H_{ad} = H_f - H_i$$

$$A_\lambda^{(1)} = i\alpha(\lambda)[H_{ad}(\lambda), H_f - H_i]$$

Gate Implementation: The commutator $[H_{ad}(\lambda), H_f - H_i]$ generates terms such as σ_i^y and $\sigma_i^z \sigma_j^y$ which account for the R_y gates and ZY interactions mentioned.

6.4 Algorithm Explanation

The algorithm combines three main ideas:

1. **Digitized Adiabatic Evolution**: The time evolution $e^{-i\int H_{ad}(t)dt}$ is Trotterized into a sequence of gates.
2. **Counterdiabatic (CD) Term**: A term H_{cd} is added to suppress excitations. A first-order approximation ($A_\lambda^{(1)}$) is used, which involves nested commutators of the adiabatic Hamiltonian, resulting in additional gates (such as Y and ZY).
3. **Bias-Field (BF) Update**: After each run, $\langle \sigma_i^z \rangle$ is measured. A bias field h_b^i is updated (using CVaR for robustness) and added to the initial Hamiltonian of the next iteration, biasing the search toward promising regions of the solution space.

6.5 CVaR Update Analysis

Conditional Value-at-Risk is defined as:

$$\mathrm{CVaR}_\alpha(X) = E[X|X \leq F_X^{-1}(\alpha)]$$

Benefits:

- Reduces sensitivity to outliers (high-energy states)
- Accelerates convergence by focusing on the best samples
- Mitigates effects of measurement noise

6.6 Role of Quantum Gates

- **Phase (R_z) and Interaction (R_{zz}) Gates**: Implement evolution under the problem Hamiltonian H_f and the adiabatic term.
- **Rotation around Y (R_y) and $Z \otimes Y$-type interactions**: These are crucial for implementing the counterdiabatic term approximation ($A_\lambda^{(1)}$). They are responsible for suppressing diabatic transitions and keeping the system in the instantaneous ground state during a faster evolution.
- **Measurement Gates**: At the end of each iteration, measurement in the Z basis provides the samples to calculate $\langle \sigma_i^z \rangle$ and update the bias fields for the next round.

6.7 Pseudocode (BF-DCQO)

1. Initialize bias fields $h_b^j = 0$ for all qubits j.
2. Define initial Hamiltonian $\tilde{H}_i = H_i + \sum_j h_b^j \sigma_j^z$ and final Hamiltonian H_f.
3. Define an evolution schedule $\lambda(t)$ from 0 to 1.
4. Repeat until convergence:
   (a) Prepare the ground state of $\tilde{H}_i$ (e.g., $|0\rangle^{\otimes n}$ if it is a transverse field).
   (b) Apply the gate sequence corresponding to **digitized and counterdiabatic evolution**: For each time step t_k:
      i. Apply adiabatic operator $e^{-i\Delta t[(1-\lambda(t_k))\tilde{H}_i + \lambda(t_k) H_f]}$ via Trotter.
      ii. Apply approximate counterdiabatic operator $e^{-i\Delta t \dot{\lambda}(t_k) A_\lambda^{(1)}(t_k)}$.
   (c) Measure all qubits, obtaining samples of bitstrings.
   (d) Calculate the energy of each sample using H_f.
   (e) Select the best samples (e.g., the α with lowest energy - CVaR).
   (f) For each qubit j, calculate $\langle \sigma_j^z \rangle$ from these best samples.
   (g) Update the bias field: $h_b^j = h_b^j + \eta \cdot \langle \sigma_j^z \rangle$ (where η is a learning rate).
   (h) Update the initial Hamiltonian $\tilde{H}_i$ with the new bias fields.
5. Return the best found bitstring.

6.8 Complexity and Resources

- **Circuit depth**: $O(L \cdot n \cdot \deg(H))$ where L is the number of Trotter steps
- **Number of qubits**: n (equal to the number of problem variables)
- **Classical iterations**: Typically tens to hundreds

7 Algorithm 4: Quantum Whale Optimization Algorithm (QWOA)

7.1 Preamble

QWOA falls within the field of quantum machine learning, more specifically in the area of metaheuristic optimization. Unlike previous algorithms, which use the evolution of a quantum

state to encode the solution, QWOA uses a **quantum representation of the population**, where each individual (a candidate solution) is represented by one or more Q-bits (qubits). The population evolution operators are inspired by quantum gates, but the algorithm is mostly executed on a classical computer, simulating quantum behavior.

7.2 Dimensionality Reduction

Before applying the QWOA algorithm, the method proposed in the paper uses a dimensionality reduction step based on *Hierarchical Agglomerative Clustering* (HAC).
This technique groups similar attributes into hierarchical clusters, allowing the selection of representative feature subsets. The goal is to reduce the search space of the optimization algorithm and improve the efficiency of the feature selection process.
After this step, the QWOA algorithm is applied to explore the space of possible feature subsets and identify those that produce the best classifier performance.

7.3 Clarification: QWOA is not a Purely Quantum Algorithm

It is crucial to distinguish: QWOA does **NOT** run on quantum hardware in the traditional sense. It is a classical algorithm that:

1. **Represents information quantumly**: Uses state vectors (α, β) for each Q-bit
2. **Simulates quantum operations**: Rotations are 2×2 matrix multiplications
3. **Maintains superposition classically**: Unlike a real quantum computer, the full population state is stored in classical memory

7.4 Mathematical Foundation of Quantum Rotation

The simulated rotation gate is implemented as:

$$\begin{bmatrix} \alpha' \\ \beta' \end{bmatrix} = \begin{bmatrix} \cos(\Delta\theta) & -\sin(\Delta\theta) \\ \sin(\Delta\theta) & \cos(\Delta\theta) \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

Rotation angle calculation in the WOA context:

$$\Delta\theta_i = \Delta\theta_{\max} \cdot \text{sign}((x^i_{\text{best}} - 0.5)) \cdot f(\text{iteration})$$

where x^i_{best} is the value of the i-th attribute in the best solution.

7.5 Algorithm Explanation

QWOA adapts the classical WOA to a quantum search space.

- **Q-bit Representation**: Each individual in the population is no longer a bitstring, but a Q-bit state, represented by amplitudes $\begin{bmatrix} \alpha \\ \beta \end{bmatrix}$. A chromosome (solution) of n attributes is represented by n Q-bits, being in a superposition of all 2^n possible subsets.
- **Observation (Measurement)**: To evaluate the fitness of an individual (feature subset), its quantum state is "observed,"collapsing the superposition into a classical state (a specific bitstring). This process is repeated to generate a classical population of solutions to be evaluated.
- **Quantum Update**: Inspired by the quantum rotation gate, the Q-bit update is performed by a rotation operator that approximates the Q-bit state to that which produced the best found solution. This guides the population toward promising regions of the search space.

7.6 Role of Gates (in simulation)

Although it does not run on quantum hardware, the algorithm simulates the behavior of gates:

- **Simulated Hadamard Gate (H)**: Used in initialization to put Q-bits in uniform superposition, ensuring initial population diversity.
- **Simulated Rotation Gate (R)**: The core of the algorithm. The rotation angle $\Delta\theta$ of each Q-bit is calculated based on the presence of the attribute in the global best solution and other WOA heuristics (spiral movement, random search). This rotation adjusts the probabilities of an attribute being selected (measured as 1) or not (measured as 0).
- **Simulated Measurement Gate**: The process of observing the Q-bit state to generate a classical solution to be evaluated. It is the interface between the quantum representation and the classical fitness evaluation.

7.7 Pseudocode (QWOA)

1. Initialize a population of m individuals, each composed of n Q-bits (for n attributes). The amplitudes (α, β) are initialized to represent uniform superposition (simulating Hadamard gates).
2. Set the maximum number of iterations.
3. For each iteration:
   (a) **Observation Phase (Classical Measurement)**: For each quantum individual, "measure"its Q-bits to obtain a classical bitstring (candidate solution). This generates a classical population of m solutions.
   (b) **Fitness Evaluation**: For each classical solution, quickly train a classifier (e.g., KNN) with the indicated feature subset. Calculate fitness $F = \alpha(\text{err}) + \beta(k)$.
   (c) **Best Solution Update**: Identify the global best classical solution (and the best neighborhood solution, if applicable).
   (d) **Quantum Population Update**: For each Q-bit of each individual:
      i. Determine the rotation angle $\Delta\theta$ based on WOA rules (encircling movement, spiral, random search) and comparison with the best solution.
      ii. Apply a simulated quantum rotation to the Q-bit, updating its amplitudes (α, β) and therefore the probability of the attribute being selected in the future.
4. Return the best feature subset (classical solution) found.

7.8 Limitations and Considerations

- **Scalability**: Explicitly storing amplitudes for n Q-bits requires $O(2^n)$ space? No! Since each Q-bit is independent, the space is $O(n \cdot m)$, where m is the population size. This is an advantage over full quantum state simulation.
- **Entanglement?**: The representation by independent Q-bits does not capture quantum correlations (entanglement) between attributes, which may limit expressiveness.
- **Simulated Quantum Parallelism**: The gain comes from the probabilistic representation, not from genuine quantum parallelism.

8 Comparative Analysis of Algorithms

8.1 Gate Architecture Comparison

Feature	**VQE**	**QAOA**	**BF-DCQO**	**QWOA**
Algorithm Type	Hybrid	Hybrid	Hybrid	Classical-Quantum
Hardware Execution	Yes	Yes	Yes	No (simulation)
Single-Qubit Gates	R_y, R_z	R_x, R_z	R_y, R_z, R_x	N/A (simulated)
Two-Qubit Gates	CX/CZ	CX (for R_{zz})	CX, CZ, Ry rotations	N/A
Optimized Parameters	$\vec{\theta}$	$\vec{\gamma}, \vec{\beta}$	$\vec{h}_b$ (and implicit)	$\Delta\theta$
Number of Layers	Variable	Fixed (p)	Fixed (L Trotter)	Iterations

Tabela 1: Comparison of gate architectures and parameters

8.2 Consolidated Benefits and Challenges

9 Conclusion

The four algorithms presented — VQE, QAOA, BF-DCQO, and QWOA — illustrate the diversity of approaches at the intersection of quantum computing with optimization and machine learning. While VQE and QAOA establish the foundation of variational methods, BF-DCQO introduces advanced physics concepts to accelerate adiabatic evolution, and QWOA demonstrates how "quantum thinking"can inspire new and efficient metaheuristics even in classical simulations. The study of the specific role of gates in each architecture reveals how different quantum resources (rotations, entanglement, interference) are exploited to solve computationally challenging problems. The comparative analysis shows that there is no single universal solution: the choice of algorithm depends on the specific problem, available resources, and hardware constraints.
As quantum technology advances, hybrid algorithms like these are expected to become increasingly relevant, combining the best of the quantum and classical worlds to tackle problems of growing complexity.

Algorithm	**Benefits**	**Challenges**
VQE	• Flexible for any Hamiltonian • Adaptable to hardware topology • Well-studied in literature	• Barren plateaus in deep ansatzes • Critical dependence on ansatz choice • Measurement cost for many terms
QAOA	• Interpretable structure • Convergence guarantee as $p \to \infty$ • Suitable for optimization problems	• Parameter optimization challenging • Depth limited by noise • Performance for small p may be modest
BF-DCQO	• Adiabatic acceleration via CD driving • Bias field loop improves convergence • Robust with CVaR	• CD term implementation is approximate • Deeper circuit than basic QAOA/VQE • Dependence on Trotterization quality
QWOA	• Classically scalable ($O(n \cdot m)$) • Better diversity than classical WOA • Does not require quantum hardware	• Not a genuine quantum algorithm • No entanglement between attributes • Performance limited by classical simulation

Tabela 2: Comparative analysis of benefits and challenges